Correct choice is all you need!

Matteo D'Agostino | [redacted] | matteo.dagostino@epfl.ch
Sajal Chaurasia | [redacted] | sajal.chaurasia@epfl.ch
Michele Smaldone | [redacted] | michele.smaldone@epfl.ch
Jacopo Ferro | [redacted] | jacopo.ferro@epfl.ch
Intelligent Agents

Abstract

The task of assisting students is challenging, as it often requires step-by-step, human-led explanations. This project aims to train a large language model (LLM) on EPFL courses to provide fast, detailed help without human involvement. We turn the 0.6 B-parameter Qwen3-Base into a compact EPFL-style tutor in three moves. (1) A single two-epoch supervised fine-tune on a 27 k English MCQA mixture (every item forced into an A–D format) raises normalised multi-token accuracy on the hidden M3_TEST set from 56.8 % to 64.5 % (+7.7 pp) and delivers similar gains on two out-of-domain splits that include code questions. (2) Direct Preference Optimisation on 37 k answer pairs adds a further almost 1 pp and noticeably cleaner explanations. (3) SmoothQuant + GPTQ compress the model to a 510 MB W4A8 checkpoint with a 9 % drop, small enough for student hardware. Swapping the default MiniLM retriever for LaBSE in our RAG branch yields an extra 0.3–1.2 pp at zero training cost. All code, data, and weights are released.

1 Introduction

The challenge of creating an AI tutor specialized for EPFL's rigorous coursework is both critical and complex: while students urgently need accessible, curriculum-aligned academic support, general-purpose chatbots like ChatGPT often fail to deliver accurate explanations for STEM domain-specific concepts and are too large for local use. This is especially true for the university level problems (Wang et al., 2024). Intelligent Tutoring Systems (ITS) such as QuadC AI Tutor (qua, 2025) and Course Assist (Feng et al., 2024) have shown the potential of delivering curriculum-aware, real-time tutoring, they are proprietary models and offer limited transparency and high costs.

To bridge this gap, we developed a tailored pipeline that transforms the open-source pre-trained Qwen3-0.6B-Base model into an efficient, EPFL-specialized tutor by first collecting student preference data to identify optimal pedagogical styles, then implementing a multi-stage training approach—leveraging Direct Preference Optimization (DPO) to align outputs with educational preferences, Supervised Fine-Tuning (SFT) for multi-choice question mastery, 4-bit quantization for local device deployment, and Retrieval-Augmented Generation (RAG) to ground responses in authoritative course materials and to avoid hallucinations.

2 Approach

2.1 DPO

Our first training stage aligns Qwen3-0.6B-Base (Yang et al., 2025) with human (or AI) preferences on STEM prompts using Direct Preference Optimization (DPO) (Rafailov et al., 2024). We build a preference dataset of triplets $\{x, y^+, y^-\}$, where x is a question, y^+ the preferred answer, and y^- a less preferred alternative. Initial data came from Milestone 1 EPFL exam questions via GPT-3.5 with targeted prompt engineering (Kojima et al., 2022), scored and filtered for quality; we then supplemented this with high-contrast pairs from public datasets (see Sec. 3.1).

DPO reparametrizes the usual RLHF objective into a single-shot likelihood loss. Define the "reward" for generating y given x as

$$r_\theta(x, y) \;=\; \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)},$$

where π_θ is the fine-tuned policy, π_{ref} a frozen reference (the base model), and β controls preference strength. The DPO objective is then

$$\mathcal{L}_{\text{DPO}}(\theta) = -\,\mathbb{E}_{(x,y^+,y^-)}\big[\log \sigma\big(r_\theta(x, y^+) - r_\theta(x, y^-)\big)\big].$$

Minimizing $\mathcal{L}_{\text{DPO}}$ increases the model's log-probability gap in favor of preferred answers without requiring a separate reward model.

2.2 MCQA

The multiple–choice component is a dense fine-tune of the public `unsloth/Qwen3-0.6B-Base` causal decoder (596 M parameters). The final pipeline is shared here for the sake of reproducibility; this section explains the choices behind each step. Every training sample is flattened into a single plain-text sequence (the same used in the `lighteval` (Habib et al., 2023) framework for evaluation) as showed in B, where the completion string is the single gold letter `"A"`–`"D"`, followed by the choice itself. No rationale text follows the letter, ensuring the answer token is the very first symbol the model must score.

The `MNLP_M3_mcqa_dataset` is loaded from the HuggingFace (HF) hub with the corresponding train/validation splits. Two mapping functions, `preprocess_train` and `preprocess_validation`, build the prompt and completion exactly as above and enforce a consistent `A-D` enumeration even when the raw item contains fewer alternatives (because they are already present in the question as per the original dataset the questions have been taken from).

Given the continuous `OutOfMemory` errors experienced while training on `GNOTO` cluster, we decided to switch to `Unsloth` framework (Daniel Han and team, 2023) in order to save on memory usage. It wraps the standard `Transformers` stack and exposes a `FastLanguageModel.from_pretrained` entry point with `full_finetuning=True` given that the small size of the base model allowed us to extensively train it with a full SFT.

Let p be the prompt token sequence and y the gold letter $\in \{A, \ldots, D\}$ followed by the choices text. The loss is the next-token cross-entropy

$$\mathcal{L}(\theta) = -\log p_\theta\big(y \mid p\big),$$

with gradients flowing through *all* the tokens. This may look weird but as described in the following sections, this training setting leaded to optimal performances, as it may have helped the model in understanding the question as well rather than just the answer.

During evaluation the prompt is broadcast across all k choices; for choice i the total log-score is

$$s_i = \sum_{t=1}^{|\mathbf{o}_i|} \log p_\theta\big(o_{i,t} \mid p, \mathbf{o}_{i,<t}\big),$$

where $\mathbf{o}_i$ is the token sequence "L_i. `option_text`". Prompt tokens are masked out of the loss so they do not influence s_i. Prediction is $\hat{\imath} = \arg\max_i s_i$.

2.3 Quantization

We used a combination of Post Training Quantization techniques. First, the Smooth Quant (Xiao et al., 2024) is applied to scale down the Activations rows (per-token quantization) and scale up the Weights columns (per-channel quantization).

$$Y = (X\text{diag}(s)^{-1})(\text{diag}(s)W) = \widetilde{X}\widetilde{W}$$

Here, X will be the input activation and W is the weight matrix. The diag(s) is estimated using the calibration dataset:

$$s_j = max(|X_j^{calib}|)^\alpha / max(|W_j|)^{1-\alpha}$$

where X^{calib} is the input activations from the calibration data samples, W is the weight matrix and α is the smoothing strength (hyperparameter). Second, the GPTQ (Frantar et al., 2023) is applied to the scaled-up weight matrices. GPTQ formulates quantization as solving the layer-wise optimization problem and uses Hessian matrices of inputs to optimally distribute quantization errors across weight parameters:

$$argmin_{\widehat{W}} ||\widetilde{W}\widetilde{X}^{calib} - \widehat{W}\widetilde{X}^{calib}||_F^2$$

where $\widetilde{W}$ are the scaled-up weights, $\widehat{W}$ are quantized weights, and $\widetilde{X}^{calib}$ is the scaled-down input activation of calibration data. Finally, for inference, the activations are scaled down using diag(s) and dynamically quantized on the fly while the weights' are statically quantized (their scales are stored in the model).

2.4 RAG

Our retrieval module indexes seven (3.1) STEM-heavy sources. OpenStax and MIT lectures document is split into ≤512-token, sentence-preserving chunks (for the cyber-sec book the question text is inlined to maximise overlap). After basic noise removal, we also coded, but did not yet benchmark, a "title-similarity" filter that drops chunks semantically distant from their course/book title, used for WIKI-stem corpus and the arxiv abstracts. Chunks are embedded once with four off-the-shelf models (`all-MiniLM-L12-v2`, `LaBSE`, `gte-small`, `gte-large`), stored in an IVF-PQ index, and queried with cosine distance; at run time

we retrieve the top-$k = 20$ passages, concatenate them with the question, and feed the context to an *unchanged* Qwen MCQA generator (∼32k-token window). No reranking, reward tuning, or giant-context models are used—by design we isolate pure retriever quality, keep compute low, and avoid the known accuracy drop that appears when LLMs ingest very long or highly noisy contexts.

3 Experiments

3.1 Data

For the MCQA SFT training run of the base model, we combined eight open-source STEM datasets; only the train and dev split of every corpus is used for the corresponding implementation stages, test splits remain untouched to avoid leakage. The resulting mixture composition is shown in (Table 1): choice-based science QA corpora (SCIQ, SCIENCEQA_TEXT_ONLY, ARC-EASY/CHALLENGE, MMLU (`auxiliary_train` only), one related to medicine QA (MEDMCQA) and one small in-house EPFL dataset obtained after the cleansing process operated on the M1 data collected among all the students. We decided to keep the format with fields `id` (debugging purpose), `dataset`, `QUESTION`, `ANSWER` (letter + option text), `CHOICES` (sequence of options without letters), `RATIONALE` (with the reasoning, where available).

Dataset	# train rows	# dev rows
SciQ (Welbl et al., 2018)	11 679	1 000
ScienceQA (Lu and et al., 2023)	506	190
ARC-Easy (Clark and et al., 2019)	2 241	567
ARC-Challenge	1 117	295
EPFL-MCQs	417	51
MedMCQA (Pal et al., 2022)	10 000	500
MMLU (Hendrycks et al., 2021)	1 569	296
Total	27 529	2 899

Table 1: Training mixture after all filters and subsampling.

To comply with the evaluation harness, every item must expose exactly four alternatives labelled `A-D`. We therefore normalise all source corpora as follows: underspecified questions offering less than 4 choices are discarded (hallucinating distractors would corrupt the label distribution); overspecified questions with $n > 4$ choices or with the correct answer at index $k \geq 4$, we keep the gold option, sample three distractors uniformly from the remaining wrong choices, shuffle the resulting 4-tuple, and re-enumerate it as A, B, C, D to avoid position bias.

The **Quantization** Calibration data is a subset of MCQA training data and English sentences from C4 (Raffel et al., 2023). Complete sentences were formed from the subset of MCQA Train data as "Question + Choices + Correct Letter + Correct Option + Reasoning". Since models will be tested on both Non-EPFL and EPFL(more comprehensive and tough) MCQA questions, hence the final Calibration dataset consists of 64 C4 English sentences, 32 EPFL MCQA Training Sentences and 32 Non-EPFL MCQA Training Sentences. All sentences have less than equal to 2048 tokens.

For the **RAG** pipeline we index seven STEM-centric knowledge bases: (i) ARXIV-MNLP (ii) ARXIV-20K, (iii) MIT-LECTURES, (iv) OPENSTAX, (v) WIKI-STEM, (vi)WIKI-STEM-CLEAN, and (vii) a 242-item cyber-security book. Histogram analysis shows (7) that some datasets had samples mean token sample falling below 512 tokens, which suggested that could be used for creating cohesive examples without losing meaning during chunking.

For **DPO**, we merge 7.5 k high-score Milestone-1 pairs (six 'best' per question by creating a customized score based on ranking criteria) with 30 k cleaned samples from `mlabonne/orpo-dpo-mix-40k-flat`, yielding 37.5 k pairs. Splits follow an 80-10-10 ratio (30 k / 3.7 k / 3.7 k train/val/test).

3.2 Evaluation methods & Baselines

MCQA We kept track of improvements by measuring the normalised accuracy (`acc_norm`) computed by `lighteval` over three datasets used as test benchmarks: THE CANTERBURY QUESTIONBANK (Sanders et al., 2013) with 654 multiple-choices questions on CS1 and CS2 topics, properly processed and uploaded on HF (`matteodagos/M3_canterbury_code`); NLP4EDUCATION (Borges et al., 2024), properly cleaned and parsed with the related script here; `matteodagos/M3_test`, a mixture of 2 551 mcqs taken from different benchmarks (ARC-EASY/CHALLENGE, MMLU and EPFL-MCQS). We used as baselines the starting base model `Qwen/Qwen3-0.6B-Base` and `Qwen/Qwen3-1.7B-Base`.

Quantized Only the Multi-token normalised accuracy is tracked and its hyperparameter tuning is done with Multi-token normalised accuracy on the MCQA Validation Set. We benchmark it against

the final M3 MCQA model on which the Post Training Quantization was performed.

RAG Evaluated on the same metrics as in MCQA. The "default RAG" baseline embeds only the HUGGINGFACE-DOCS corpus with `thenlper/gte-small` (identical IVF-PQ, $k = 20$) and feeds the result to the same Qwen fine-tuned MCQA generator.

DPO Rewards/accuracy from `HF DPOTrainer` measures the fraction of pairs where $r_\theta(y^+) > r_\theta(y^-)$; Δ-accuracy from `LightEval` replaces the reference by the untuned base model, so $\Delta = 0$ for an unchanged model.

3.3 Experimental details:

MCQA We initially adopted a curriculum strategy: one epoch on the mixed MCQA corpus described in Table 1, followed by 3 epochs on the EPFL_MCQS alone. Validation loss stopped decreasing after the first stage and accuracy remained unchanged, indicating that the model had already saturated the task. We therefore reverted to a single–phase fine-tune on the full mixture.

Our best MCQA checkpoint is obtained with the following schedule. We keep a `per_device_train_batch_size` of 4 and accumulate gradients for 16 steps, yielding an effective batch of 64 sequences. Optimisation runs for exactly two epochs with a peak learning-rate of 2×10^{-5} under a linear decay schedule and a warm-up fraction of 5 %. We apply weight decay $= 0.01$ and clip the global gradient norm at 1.0. Training is performed in `bf16` mixed precision, and gradient checkpointing is enabled to fit the model comfortably on a single `A100`. Compute resources were quite limited, so we performed only manual tuning around hyperparameters: wider grid or Bayesian search was infeasible. We then concluded that revisiting the *input representation* offered a higher return on effort than further hyper-parameter sweeps and we therefore re-structured the training data (first by pre-pending, then by appending the available rationales) and measured the effect of each variant (results in the next subsection).

DPO Direct Preference Optimisation trains Qwen3-0.6B-Base to favour "chosen" over "rejected" answers without a separate reward model.

Training. Unsloth LoRA (rank 16, α16) finetunes all attention and MLP blocks for two epochs with cosine LR 3×10^{-6}, 3 % warm-up, effective batch 16 tokens, bfloat16 precision and 4-bit weight quantisation under base model but merged at 16 bit with fresh base model; gradient checkpointing keeps memory usage modest.

Quantized We quantize the final M3 MCQA model. We use the LLM Compressor library (AI and vLLM Project, 2024) to use Smooth Quant (smoothing strength 0.7) followed by GPTQ to quantize the weights to INT4 Static Symmetric Channel and the activations to INT8 Dynamic Symmetric Token. We had performed hyperparameter tuning on M2 MCQA Model and quantized the final M3 MCQA model with the best configuration. We had tuned on different sizes of (C4 English Sentences + EPFL MCQA Train Sentences) calibration data and added an additional 32 Non-EPFL MCQA Train sentences to the calibration dataset (see Appendix B for results), varied the smoothing strengths (see Plot 1) and used different quantization recipes (see Table 4).
We also packed every eight int4 weights into one int32 containers using bitwise operations. We enabled LLM compressor to pack the W4A8 INT models since at present, only W4 INT and W8 INT models are packed by it. Packing of Asymmetric quantized weights is not supported by the library hence, its size is bigger. The 'lm_head' (language modeling head) layer is not quantized in all of the quantization experiments since quantization introduces approximation errors that can drastically affect final predictions, making the output less reliable.

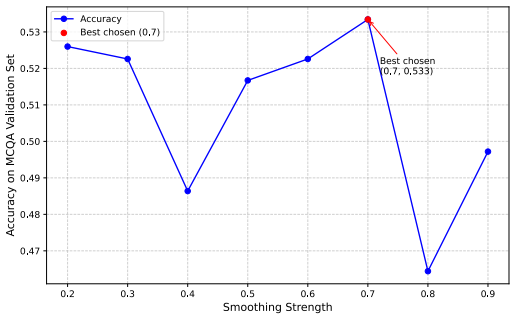

Figure 1: Val MCQA accuracy vs. Smoothing Strength

RAG Each corpora is lightly cleaned, and embedded once with each model {`MiniLM`, `LaBSE`, `gte-small`, `gte-large`}. Embeddings are stored in a single-shard IVF-PQ. At inference, we retrieve the top-20 chunks, concatenate with the question, and pass the ∼3k-token prompt to the frozen 0.6B-parameter Qwen MC-QA finetuned model. No reranking, reward tuning, or long-

context readers are used; experiments run on a single A100 40GB GPU, indexing ≤1 hour per corpus and ≤0.5s/query at test time. A prototype "title-similarity" pre-filter was coded but disabled after it removed too many samples from WIKI-STEM and ArXiv abstracts. all reported numbers and explorations are present in the appendix (7).

3.4 Results

MCQA The results obtained over the three different benchmarks are summarised in Table 2 and 3. It is worth highlighting that our best MCQA checkpoint outperforms the Qwen3-0.6B-Base baseline by almost **+8pp** in normalised multi-token accuracy. The gain extends to code-oriented questions, a domain scarcely represented in the training mix (only a few items in EPFL_MCQs), indicating that the model has generalised beyond the dominant natural-language STEM content.

Model	M3_test	M3_NLP4 education	M3_Canterbury Code
Qwen3-0.6B-Base	0.6131	0.4532	0.3845
Qwen3-1.7B-Base	0.7428	0.5528	0.5216
M3_mcqa_model	0.6476	0.4892	0.4367

Table 2: Single-token normalised accuracy (`acc_norm`)

Model	M3_test	M3_NLP4 education	M3_Canterbury Code
Qwen3-0.6B-Base	0.5676	0.4140	0.3592
Qwen3-1.7B-Base	0.7154	0.4938	0.4933
M3_mcqa_model	0.6445	0.4763	0.4188
M3_quantized_model	0.5527	0.4002	0.3487
M3_mcqa_model + RAG	0.6350	0.4730	0.4277

Table 3: Multi-token normalised accuracy (`acc_norm`).

Quantized The Quantized model achieved 55.27% while the MCQA model achieved 64.45% accuracy. Hence, we see a ∼9% decline probably due to the rounding errors introduced by PTQ techniques. Also, MCQA is a very sensitive task where the quantized model has to preserve STEM information and formatting as well.

RAG Across all three evaluation splits (Table 7), the multilingual **LaBSE** retriever is consistently the best drop-in swap. On full corpora, it edges the MiniLM baseline by ~0.3–1.2 pp: ARXIV-64K 63.50 ± 0.95 vs. 63.27, MIT 63.15 ± 0.96 (raw tie but higher *acc_norm*), and WIKI-STEM 63.39 ± 0.96 vs. 62.21. A "clean" 100-question slice amplifies the gap—**72%** accuracy (+9 pp). In contrast, **gte-large** collapses on the combined corpus (≤ 42%), pointing to chunk-length overflow in the generator context. Corpus quality also matters: HuggingFace docs bottom out at 61%, while trimming ArXiv to 20k abstracts slightly *lowers* accuracy, confirming that aggressive filtering without being careful of the preprocessing can hurt more than it helps.

Table 4: Quantization Scheme vs Val MCQA Accuracy vs Model Size (with A8 INT Sym. Token in all)

S. Strength	Weight Quant Scheme	Acc.	Size (MB)
None	W4 INT Sym. Channel	0.4567	507.534
0.7	W4 INT Sym. Channel	**0.5334**	507.534
0.7	W4 INT Asym. Channel	0.4514	717.860
0.7	W4 INT Sym. Group128	0.5019	513.440
0.7	W4 INT Sym. Group256	0.5053	510.160
0.7	W4 INT Sym. Tensor	0.2416	506.880
0.7	W8 INT Sym. Channel	**0.5381**	717.530

DPO On the validation split rewards/accuracy reaches **0.74**; on the 3.7 k test pairs Δ-accuracy is **0.715**. Interpolating the LoRA with the MCQA SFT weights improves normalised MCQA accuracy on `matteodagos/M3_test` from 0.564 (pure DPO) to **0.648** at $\alpha = 0.5$, while adding the LaBSE RAG retriever lifts it to 0.596.

4 Analysis

4.1 Quantization

We analyzed the influence of the calibration set (Williams and Aletras, 2024) where increasing the size of calibration set yielded reduction in Val MCQA accuracy. A slight increase in accuracy(∼ 2%) happened when we added 32 Non-EPFL MCQA Training Sentences suggesting that calibration on the examples which will be used during inference is useful. We also analyzed the effect of smoothing between activations and weights which gave an improvement of 8%, which highlights its advantage of transferring qunatization difficulty from activations to weights. Tensor wise Quantization of weights yielded a very low val accuracy(∼ 24%) although it gave a more compressed model due to usage of one weight scale per weight matrix however, the rounding error via quantization is also very high. We also tried 8 bit INT quantization of weights but it yielded only (∼ 0.46%) increment in validation accuracy hence, we decided to use the 4 bit INT quantization for weights.

4.2 MCQA

1) *Rationales*. Pre-pending free-form explanations (where present) to the prompt drops `acc_norm` $= 0.6158$ over M3_TEST, while appending them to the completion yields 0.6296. The evaluation harness scores the *first* answer token; any additional tokens dilute the probability mass, so longer prefixes are detrimental.

2) *Extra AQUARAT*. Adding $10\,000$ math items to the training dataset (`matteodagos/MNLP_M3_mcqa_dataset_extended`) did not improve accuracy, confirming that Qwen3 already covers the underlying STEM knowledge and mainly needs guidance on the MCQA output format: the second of the three-stages pre-training (Team, 2025) confirms that.

3) In the bestowed bonus days we also tried a Direct Preference Optimisation (DPO) separate training run for the mcqa model, by treating the correct `letter+option` span as "chosen" and a random distractor (or the same option with a prepended rationale) as "rejected". The DPO-fine-tuned model reached `acc_norm` $= 0.6284$ on M3_TEST: competitive but still below the plain SFT baseline of 0.6445, so we kept SFT as our primary submission.

4.3 RAG

LaBSE's advantage is modest ($\approx$0.3–1.2 pp) yet unfailingly relaiable across every STEM corpus, confirming that multilingual pre-training copes better with technical jargon than purely English MiniLM. However, more text does not equal to better performances: adding huge, heavily-chunked sources (MIT lectures, OpenStax) left accuracy flat at $\sim$63%, showing that distractor noise cancels any coverage gain. The best normalised scores come from domain-aligned sets (ArXiv, MIT); reinforcing that retrieval quality hinges on corpus relevance, not size. Finally, this echoes prior work and current questions about RAG retreival: beyond $\sim$10k docs (or $\sim 50M$ tokens) (red, 2025) or 8k-token prompts (Team, 2025), retrieval or context length dilutes attention and accuracy falls.

5 Ethical considerations

Even if Qwen3 has been trained on 119 languages (Team, 2025), our models are fine-tuned only on English, so accuracy falls when the same model is queried in French (the other teaching language at EPFL) or in low-resource languages, like Urdu and Swahili. We can adapt it by (i) translating existing MCQs to bootstrap training and (ii) adding a small, manually vetted dev set per language. As a tutor the model gives students low-cost feedback and helps instructors draft or vet questions. The same functionality also enables cheating in remote exams and could bias test construction if educators treat the model's difficulty estimates as ground truth, disadvantaging non-native speakers. To handle and mitigate the mentioned problems we therefore propose: rate-limits and on-campus deployment during assessments; watermarking to flag model-generated answers; broader, multilingual training data plus periodic audits on minority language slices; mandatory human review of generated items and rationales.

6 Conclusion

In this project, we implemented an educational chatbot capable of performing well on Stem MCQAs and giving helpful responses. We worked on democratizing the AI Tutor by quantizing the model although it resulted in some performance drop. We were constrained by memory, quality and quantity of data and small model, however we still demonstrated that compact models can achieve strong performance on specialized academic tasks through targeted training approaches, challenging the assumption that larger models are always necessary. For RAG, curating contextually meaningful, benchmark-relevant chunks is more important than huge generic data or heavy embedders. High-quality preference pairs enable stable DPO on a 0.6 B model; a light DPO "nudge" boosts MCQA accuracy, but heavier mixing shows diminishing returns. Full logs and additional ablations are available in the repository as described in the README file. Also, with the MCQA SFT training we discovered that a good quality, well formatted and rigid dataset makes the difference at this scale, more than trying to induce reasoning by inserting the rationales into the completions, here's why "Correct choice is all you need!".

References

2025. Does rag have a scaling problem? https://www.reddit.com/r/ArtificialInteligence/comments/1g431u3/does_rag_have_a_scaling_problem/. Reddit discussion, accessed 11 Jun 2025.

2025. QuadC AI Tutor. https://www.quadc.io/features-ai-tutor-for-students. Accessed: 2025-06-11.

Red Hat AI and vLLM Project. 2024. LLM Compressor.

Beatriz Borges, Negar Foroutan, Deniz Bayazit, Anna Sotnikova, Syrielle Montariol, Tanya Nazaretzky, Mohammadreza Banaei, Alireza Sakhaeirad, Philippe Servant, Seyed Parsa Neshaei, Jibril Frej, Angelika Romanou, Gail Weiss, Sepideh Mamooler, Zeming Chen, Simin Fan, Silin Gao, Mete Ismayilzada, Debjit Paul, Philippe Schwaller, Sacha Friedli, Patrick Jermann, Tanja Käser, Antoine Bosselut, EPFL Grader Consortium, and EPFL Data Consortium. 2024. Could chatgpt get an engineering degree? evaluating higher education vulnerability to ai assistants. *Proceedings of the National Academy of Sciences*, 121(49):e2414955121.

Peter Clark and et al. 2019. Ai2 reasoning challenge.

Michael Han Daniel Han and Unsloth team. 2023. Unsloth.

Ty Feng, Sa Liu, and Dipak Ghosal. 2024. Courseassist: Pedagogically appropriate ai tutor for computer science education.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. Gptq: Accurate post-training quantization for generative pre-trained transformers.

Nathan Habib, Clémentine Fourrier, Hynek Kydlíček, Thomas Wolf, and Lewis Tunstall. 2023. Lighteval: A lightweight framework for llm evaluation.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213.

Lei Lu and et al. 2023. Scienceqa.

Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. 2022. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Proceedings of the Conference on Health, Inference, and Learning*, volume 174 of *Proceedings of Machine Learning Research*, pages 248–260. PMLR.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2023. Exploring the limits of transfer learning with a unified text-to-text transformer.

Kate Sanders, Ahmadzadeh Marzieh, Tony Clear, Stephen Edwards, Mikey Goldweber, Chris Johnson, Raymond Lister, Robert Mccartney, Elizabeth Patitsas, and Jaime Spacco. 2013. The canterbury questionbank: building a repository of multiple-choice cs1 and cs2 questions. pages 33–52.

Qwen Team. 2025. Qwen3 technical report.

Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun R. Loomba, Shichang Zhang, Yizhou Sun, and Wei Wang. 2024. Scibench: Evaluating college-level scientific problem-solving abilities of large language models.

Johannes Welbl, Pontus Stenetorp, and Sebastian Riedel. 2018. Sciq — a large dataset of science questions.

Miles Williams and Nikolaos Aletras. 2024. On the impact of calibration data in post-training quantization and pruning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, page 10100–10118. Association for Computational Linguistics.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2024. Smoothquant: Accurate and efficient post-training quantization for large language models.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 technical report.

A Contributions

We coordinated for project results, technical discussions and tasks during the course of the semester. We helped each other but had main responsibilities as defined in the previous reports.
Matteo: Worked on data retrieval and data cleaning for SFT, benchmarks generation, training of MCQA model and report writing.
Sajal: Primarily worked on the Quantization, assisted in DPO training and contributed in Report Writing.
Michele: Worked on RAG and report writing.
Jacopo: Worked on DPO, ablation studies, relative data, report writing.

B Appendix

AI Usage:

We used chatbots such as ChatGPT, Deepseek and Claude for different parts of our project. We used them to give a preliminary version of the code for training LLMs for DPO and Supervised Fine-Tuning, which we rigorously verified step by step and modified for our use-case. We also used them for correcting our grammatical mistakes in our project reports as well as to make some of the paragraphs more coherent. We prompt the chatbot to be more formal and concise when we use it to shorten some of the paragraphs for our reports. The LLM Chatbots were not very useful when we asked them the code to measure peak VRAM usage when using vLLM as the inference engine. ChatGPT kept on suggesting to use torch.cuda.max_memory_allocated() or nvidia-smi which did not work as intented but it did not suggest the vllm memory profiling tool which one can easily find in vllm documentation. We then used the wrapper provided in the first steps of the project for the cleaning and formatting process of multiple choice questions coming from M1, with an expert prompting approach in order to formulate a summarizing improved rationale for each data point by starting from 10 random rationales provided by the students.

Calibration Data Mixtures:

We check the effect of using different calibration datasets on Quantized Model's performance. The quantization method is held same as Smoothing with Strength 0.7 and then GPTQ to get W4 INT Static Symmetric Channel and A8 INT Dynamic Symmetric Token quantization.

Table 5: Calibration Mixtures vs Val MCQA Accuracy

Calibration Data Configuration	Accuracy
64 C4 En + 32 EPFL	0.5161
128 C4 En + 64 EPFL	0.4889
256 C4 En + 128 EPFL	0.4774
512 C4 En + 256 EPFL	0.5028
64 C4 En + 32 EPFL + 32 Non-EPFL	**0.5334**

Format used for questions in training the best MCQA model:

```
The following are multiple choice questions (with answers) about knowledge and skills in advanced master-level STEM courses.

{QUESTION}
A. {choice_1}
B. {choice_2}
C. {choice_3}
D. {choice_4}
Answer:
```

RAG Exploration

Table 6: Corpus-level statistics for analyzing rows length distributions.

Corpus	# Samples	Mean tok.	Median tok.	Max tok.	Total tokens	1–128
`arxiv-mnlp-cleaned`	63 357	230.0	223	914	14 574 180	7 308
`arxiv-20k`	20 000	267.0	254	864	5 340 283	0
`mit-mnlp-cleaned`	1 444	6 615.7	6 938	26 989	9 553 100	1
`openstax-mnlp-cleaned`	9	348 223.0	339 775	532 162	3 134 007	0
`cyber-mnlp`	242	62.5	58	109	15 136	242
`WIKI-STEM (amene-gafsi)`	70 859	412.0	426	632	29,195,461	0

Table 7: Results on various M3 test-splits for different 3 datasets & exploration on TAs test split. with top-k 20 and cosine similarity and no chat template.

Dataset	Embedder + matteodagos/M3_mcqa_model	Norm. Accuracy (± StdErr)
Dataset test split: M3_test		
Smxldo*`/arxiv-mnlp-cleaned`	all-MiniLM-L12-v2	0.6268 ± 0.0096
	LaBSE	0.6350 ± 0.0095
	LaBSE (100 samples)	0.7100 ± 0.0456
`*/arxiv-20k`	LaBSE	0.6241 ± 0.0096
	LaBSE (100 samples)	0.7000 ± 0.0461
`*/mit-mnlp-cleaned`	LaBSE	0.6123 ± 0.0096
	LaBSE (100 samples)	0.7100 ± 0.0456
`*/mit-chunked-512`	LaBSE (100 samples)	0.6300 ± 0.0485
`*/openstax-mnlp-cleaned`	LaBSE (100 samples)	0.6500 ± 0.0479
`amene-gafsi/WIKI-STEM-CORPUS`	LaBSE	0.6221 ± 0.0096
	LaBSE (100 samples)	0.7200 ± 0.0451
Dataset test split: M3_canterbury_code		
`*/arxiv-mnlp-cleaned`	all-MiniLM-L12-v2 (100 samples)	0.41 ± 0.0494
	LaBSE (100 samples)	0.43 ± 0.0498
	LaBSE	0.4277 ± 0.0191
Dataset test split: M3_nlp4education		
`*/arxiv-mnlp-cleaned`	all-MiniLM-L12-v2 (100 samples)	0.47 ± 0.0502
	all-MiniLM-L12-v2	0.4795 ± 0.0107
	LaBSE	0.4730 ± 0.0107
Dataset test split: MNLP_STEM_mcqa_evals (TAs debug dataset, test split) with Qwen Base as generator		
`m-ric/huggingface_doc`	thenlper/gte-small	0.3974 ± 0.0176
`*/mit-mnlp-cleaned`	thenlper/gte-small	0.3831 ± 0.0175
`*/arxiv-mnlp-cleaned`	all-MiniLM-L12-v2	0.4117 ± 0.0177
	thenlper/gte-small	0.4052 ± 0.0177
	all-MiniLM-L12-v2 (70 samples)	0.3429 ± 0.0571
	LaBSE (70 samples)	0.3714 ± 0.0582
`*/openstax-mnlp-cleaned`	thenlper/gte-small	0.3688 ± 0.0174
`*/combined`	all-MiniLM-L12-v2	0.3935 ± 0.0176
	gte-small	0.3831 ± 0.0175
	gte-large	0.3792 ± 0.0175